

ABM BUSINESS LOGIC BIBLE

SECTION 4 — INTEGRATIONS, APIs, FAILURE MODES & ENGINE

EPISTEMOLOGY

Full 5-Whys Excavation · 12 Domains · The External Boundary AND the Reality-Confidence Spine

Sections 1–3 defined the engine's principles, entities and modules. Section 4 defines its **boundary with the outside world** and, in this revision, the **epistemic layer that governs how it decides what reality to trust**. External systems are not neutral sources of truth — they are independent actors with incentives, biases, blind spots, costs and outages. The headline domain **INT-SIG** is therefore reframed: it no longer merely collects intelligence, it continuously constructs an **evolving causal model** of each target (what is changing, why, who drives it, who resists, how power flows, how it connects to Decimal) — and every belief it holds is governed by the new **EPIS** layer and its **Reality Confidence Model**.

Domains	12	5-Whys chains	26
New rules	~152	Spine	EPIS / Reality Confidence

#	Domain	Scope	Rules
4.0	EPIS	Engine Epistemology — Reality Confidence Model (the governing spine)	18
4.1	INT-SIG	Account Intelligence Acquisition + 8 reasoning streams (causal model)	34+23
4.2	INT-ENR	Enrichment (Apollo/Clearbit/Clay) — fallback chain, cost gating	7
4.3	INT-LIN	Heyreach LinkedIn — platform-policy limits, ban-risk circuit breaker	7
4.4	INT-EML	Email + deliverability — SPF/DKIM/DMARC, warmup gating	8
4.5	INT-CRM	HubSpot sync — idempotency, authority split, conflict resolution	6
4.6	INT-SLK	Slack notifications — brief contract, rate limiting	4
4.7	INT-ANL	Metabase analytics — read-replica, no write-back	4
4.8	INT-ORC	n8n orchestration — no-business-logic boundary, idempotent steps	7
4.9	API	Service contracts — 5 endpoints, auth, versioning, rate limits	10
4.10	FAIL	Retry / DLQ / circuit-breaker matrix + failure learning	9+2
4.11	OBS	Observability + PDPL / security / compliance	7

4.0 EPIS — ENGINE EPISTEMOLOGY & THE REALITY CONFIDENCE MODEL

This domain governs the other eleven. Once the engine makes high-value strategic recommendations, the limiting factor is no longer intelligence *acquisition* but **confidence in the intelligence itself**. EPIS is the layer that answers: how do we know something is true, how certain are we, what evidence supports or contradicts it, and what would change our mind? Every fact INT-SIG stores, every hypothesis it generates, and every recommendation it emits is stamped by the Reality Confidence Model defined here.

THE SPINE — REALITY CONFIDENCE MODEL (RCM)

Every belief the engine holds carries an **RCM stamp**: a confidence score plus its evidential basis (supporting evidence, contradicting evidence, source reliability, source incentive-bias, coverage, and decay state). The RCM is the **governor** on the INT-SIG reasoning streams (4.1): no hypothesis, influence score, tension score or relevance score is surfaced to a human or fed to scoring without an RCM stamp, and low-confidence beliefs are labelled, de-weighted, or withheld — never presented as fact.

Why #1: Why does the engine need an epistemology layer at all, beyond source-trust weighting?

Because source-trust weighting models how reliable a source is, but not why it publishes, what the engine cannot see, what its beliefs cost to acquire, how fast they decay, or how confident the engine should be overall. Recommendations made without that frame carry hidden, uncalibrated confidence — the failure mode Principle 3 warns of.

Why #2: Why separate a source's reliability from its incentive-bias?

Because a source can be highly reliable yet highly biased: a bank press release is accurate about the fact of an event but incentivized to frame it as success. Collapsing the two makes the engine trust a biased framing as if it were neutral truth, corrupting every hypothesis built on it.

Why #3: Why must the engine model what it cannot see (coverage / blind spots)?

Because the events that drive enterprise purchases — budget meetings, procurement rejections, board disagreements, vendor dissatisfaction — are largely invisible. An engine that scores only on visible facts is confidently wrong whenever the decisive action is unobserved. Confidence must fall as coverage falls.

Why #4: Why attach an acquisition cost and a decay curve to information itself?

Because information is neither free nor permanent. Enriching every account and researching every executive wastes scarce capital (Scarcity Economics), and a generic freshness TTL mis-ages everything equally — a job posting and an organization's identity do not decay at the same rate. Both cost and semantic half-life must be explicit.

Why #5: Why does confidence in intelligence become the binding constraint as the engine matures?

Because LinkedIn can be wrong, Apollo stale, press releases biased, executives can misrepresent priorities, vendors exaggerate success, and humans misinterpret signals. Once recommendations are high-stakes, the question shifts from 'what do we know?' to 'how sure are we, and what would change our mind?' — which only an explicit RCM can answer.

Root cause: The engine must maintain an explicit, falsifiable model of its own certainty — per fact, per account, per recommendation — governing what it trusts, what it spends to learn, and what it admits it does not know.

Component	Governs	Output
Source incentive-bias model	Why a source publishes (success/sell/enforce/attention)	Reliability Score ⊕ Incentive-Bias Score (separate axes)
Intelligence coverage model	What % of the buying system is observable	Coverage Score → caps confidence
Knowns / Unknowns / Critical-Unknowns	What every recommendation must declare	Explicit uncertainty ledger

Component	Governs	Output
Value-of-information gating	Whether to spend to acquire intelligence	Acquire iff $\Delta\text{DecisionValue} > \text{AcquisitionCost}$
Semantic half-life	Per-category decay, not generic TTL	Category-specific decay curve
Per-subsystem trust	Trust measured per domain, not globally	Trust + Error + Override rate per subsystem
Reality Confidence Model	Every belief's overall certainty + basis	RCM stamp (the spine)

New rules

Rule ID	Specification
EPIS-SRC-01	Every source carries TWO independent scores: Reliability (accuracy track record) and Incentive-Bias (motive to distort — success/sell/enforce/attention). They are never collapsed into one number.
EPIS-SRC-02	Incentive-Bias is applied when a source's framing (not just its facts) feeds a hypothesis; high-bias framing cannot raise a hypothesis's confidence on its own.
EPIS-COV-01	Every account carries an Intelligence Coverage Score = observable share of its buying system (executives/vendors/signals visible vs budget/politics/opposition/priorities invisible).
EPIS-COV-02	Coverage caps confidence: low coverage mechanically reduces the RCM confidence of any account-level recommendation; the engine cannot be more certain than its coverage allows.
EPIS-COV-03	Every recommendation declares Knowns, Unknowns and Critical Unknowns (the unknowns that would most change the decision), feeding Principle 3's uncertainty-reduction loop.
EPIS-VOI-01	Every intelligence action carries an Acquisition Cost (API + human + compute + time).
EPIS-VOI-02	Information is acquired only if Expected Decision Improvement > Acquisition Cost (value-of-information gate); below threshold, the engine deliberately stays uninformed and records why.
EPIS-HALF-01	Every intelligence category has an expected decay curve (semantic half-life), e.g. job posting ~30d, conference speech ~180d, vendor contract ~365d, organization identity ~5y — replacing generic freshness TTL.
EPIS-HALF-02	Decay state is part of the RCM stamp; an aged belief loses confidence on its own curve, not a global clock.
EPIS-TRUST-01	Trust is measured PER subsystem (intelligence, enrichment, scoring, AI, automation, data quality), each with Trust Score + Error Rate + Human-Override Rate; trust loss in one does not collapse the others.
EPIS-TRUST-02	A subsystem whose override rate or error rate breaches threshold is flagged low-trust; its outputs are de-weighted and surfaced for review, not silently trusted.
EPIS-RCM-01	Every fact, hypothesis and recommendation carries an RCM stamp: confidence + supporting evidence + contradicting evidence + source reliability/bias + coverage + decay state.
EPIS-RCM-02	RCM governs the INT-SIG reasoning streams (4.1): no SIG-HYP/POWER/TENSION/RELEVANCE/VSAT output is surfaced or fed to scoring without an RCM stamp.
EPIS-RCM-03	Confidence is calibrated and falsifiable: every confidence figure links to the evidence that produced it and the evidence that would lower it ('what would change our mind'). Free-floating AI-generated percentages are rejected.
EPIS-RCM-04	Low-confidence beliefs are labelled, de-weighted or withheld — never presented to a human as established fact (anti-false-confidence, ties to Principle 3 and Module 0 hallucination governance).
EPIS-RCM-05	The engine never fabricates confidence: absence of evidence is recorded as a Critical Unknown, not filled with an inferred value.
EPIS-ETH-01	Reasoning about individuals (influence, belief, tension) is bounded to PUBLICLY observable signals with sources; the engine never infers private, protected or sensitive attributes of a named person, and operates within PDPL (Constraint Hierarchy top tier).
EPIS-ETH-02	Political/power/tension intelligence is competitive analysis of public signals only; any inference lacking a public-source basis is recorded as ~inferred at capped confidence and excluded from automated action.

4.1 INT-SIG — ACCOUNT INTELLIGENCE ACQUISITION ENGINE

INT-SIG is not a feed reader. It continuously produces, per target account, a **living Master Intelligence Dossier** and resolves the connection path back to Decimal — automating the BD methodology proven on SNB, Riyadh Bank and Al Rajhi. Eight sub-streams harvest into one account intelligence graph; the headline stream SIG-PATH is the acquisition target, since a signal with no warm path is unactionable.

Sub-stream	Acquires (Riyad Bank example)	Cadence
SIG-NEWS	News, announcements, press releases, public circulars	Hourly
SIG-EVENT	Events, seminars, conferences, awards, speaking slots	Daily
SIG-FIN	Financial position, quarterly results, annual report, ratings	On-publish + quarterly
SIG-REG	SAMA/CMA regulatory circulars, licences, compliance moves	Daily
SIG-EXEC	Full C-suite dossier: bio, tenure, public speeches/quotes, moves	Weekly + event
SIG-VENDOR	Vendor relationships + activity (Efignce at Riyad) + gap	Monthly + event
SIG-SUBS	Subsidiaries and their activity	Monthly + event
SIG-PATH	Connection paths to Decimal (LinkedIn degree, mutual, shared node)	Continuous

SIG-PATH — connection-to-Decimal (resolved against the full set)

Priority ladder: P1 LinkedIn 1st-degree (Decimal employee ↔ target) > P2 mutual connection via a Decimal person > P3 shared node (ex-employer / alumni / vendor / board) > P4 generalized connector bridge (KPMG/Mastercard/Deloitte/Tarabut/Fintech-Saudi) > P5 cold email pattern. Resolved against **both** Decimal's ingested employee network and the generalized connector registry.

Hard-coded tradecraft: cross-group power-node detection (one exec across ≥2 entities), IT-subsiary gatekeeper sequencing (route through the EJADA-equivalent before C-suite), path decay TTL, executive-departure-as-dual-signal, and strict anti-fabrication (never invent a path or a contested name).

INT-SIG carries 34 rules (SIG-PATH 10 · SIG-EXEC 5 · SIG-VENDOR 4 · SIG-SUBS 2 · SIG-NEWS 2 · SIG-REG 2 · SIG-FIN 1 · SIG-EVENT 1, plus the consolidated index). Full 5-Whys for all eight sub-streams is documented in the standalone Checkpoint 4.1 PDF and is incorporated here by reference to keep this unified document navigable.

INT-SIG carries 34 acquisition rules (SIG-PATH 10 · SIG-EXEC 5 · SIG-VENDOR 4 · SIG-SUBS 2 · SIG-NEWS 2 · SIG-REG 2 · SIG-FIN 1 · SIG-EVENT 1, plus the consolidated index). Full 5-Whys for all eight acquisition sub-streams is in the standalone Checkpoint 4.1 PDF, incorporated here by reference.

4.1.R INT-SIG REASONING STREAMS — FROM FACTS TO A CAUSAL MODEL

Humans do not decide from facts; they decide from **hypotheses about facts**. A bare fact ('new CTO hired') is inert until the engine reasons about why it happened, who drives it, who resists, whether the incumbent is vulnerable, and whether it even matters to Decimal. These eight reasoning streams turn INT-SIG from a descriptive repository into an evolving causal model. Every output is governed by the EPIS Reality Confidence Model (4.0) — stamped, calibrated, and bounded to public signals.

Stream / Entity	Turns this fact...	...into this reasoning
SIG-HYP	New CTO hired	Competing explanations w/ calibrated confidence (transformation 60% / succession 25% / regulatory 15%)
ACCESS_STRENGTH	P1 LinkedIn path exists	Whether the path is usable (freshness, trust, reciprocity, intro likelihood)
SIG-VSAT	Backbase installed	Is the incumbent vulnerable? → Displacement Probability
EXEC_BELIEF_TIMELINE	2025 keynote quote	Belief trajectory 2023→2025 → predicted buying direction
SIG-POWER	Org chart titles	Who actually influences decisions → Influence Score ≠ title score
SIG-TENSION	CTO + Compliance + Procurement signals	Internal friction → Organizational Friction Score → deal velocity
SIG-MOBILITY	CTO leaves SNB, joins Riyadh	Opportunity Transfer Event (relationship + knowledge follows the person)
SIG-RELEVANCE	Bank sponsors football tournament	Strategic Relevance Score → noise filtered, signal kept

WHY THESE BELONG TO INT-SIG, NOT A SEPARATE MODULE

Each stream is a reasoning operation over already-acquired facts, so it lives where the facts live. But each also depends on EPIS: a hypothesis without an RCM stamp is just a guess; an influence or tension score without a public-source basis is profiling. The reasoning streams produce the causal model; EPIS keeps that model honest.

Shared 5-Whys spine (reasoning cluster)

Why #1: Why store competing explanations instead of the single most-likely cause?

Because committing early to one explanation is how the engine becomes confidently wrong. Holding ranked hypotheses with calibrated confidence keeps reasoning causal and revisable as new evidence arrives (SIG-HYP).

Why #2: Why distinguish access from connection, and satisfaction from presence?

Because a connection that cannot be activated is not a path, and an incumbent that is present but resented is the real opportunity. Modeling existence without usability/quality overstates both reachability and competitive difficulty (ACCESS_STRENGTH, SIG-VSAT).

Why #3: Why track belief trajectories, influence and tension rather than snapshots and titles?

Because enterprise decisions emerge from how thinking evolves, who truly drives it, and the friction between factions — none visible in a static org chart or a single quote. Trajectories and friction predict buying behavior and deal velocity (EXEC_BELIEF_TIMELINE, SIG-POWER, SIG-TENSION).

Why #4: Why model the market as a network rather than a set of separate account dossiers?

Because executives, consultants, vendors and board members move between targets, carrying relationships and knowledge with them. A departure is not just a new signal at one account — it is an opportunity-transfer event across the network (SIG-MOBILITY).

Why #5: Why add a dedicated relevance gate when scoring already prioritizes?

Because thousands of facts arrive and most are noise; relevance is a distinct judgment (does this touch Decimal's solutions, the account's initiatives, the current narrative or opportunity?) that must filter intelligence

before it dilutes the model (SIG-RELEVANCE).

Root cause: INT-SIG must continuously construct and revise a causal model of each target — explanations, access, incumbent vulnerability, belief evolution, real influence, internal friction, network mobility, and strategic relevance — all governed by EPIS so the model never outruns its evidence.

New rules

Rule ID	Specification
SIG-HYP-01	Every material signal stores competing explanations (hypotheses), each with a calibrated, RCM-stamped confidence; the engine reasons causally, not descriptively.
SIG-HYP-02	Hypothesis confidences are evidence-linked and falsifiable (EPIS-RCM-03); they are revised as new evidence arrives and never presented as settled fact.
SIG-HYP-03	Hypotheses feed Strategy reasoning (Principle 5) as explicit causal premises, not hidden assumptions.
ACCESS-01	ACCESS_STRENGTH is modelled separately from path existence (SIG-PATH), scored on relationship freshness, trust, reciprocity and introduction likelihood.
ACCESS-02	Effective path rank = path tier × access strength; a P3 with strong access can outrank a P1 with weak/stale access. Sender assignment uses effective rank.
ACCESS-03	Access strength is sourced from observable interaction signals only; it never asserts a private relationship state without basis (EPIS-ETH-01).
VSAT-01	SIG-VSAT tracks incumbent-vendor satisfaction (negative reviews, implementation delays, executive complaints, vendor turnover, replacement chatter) and outputs a Displacement Probability per incumbent.
VSAT-02	Displacement Probability feeds Effective Opportunity alongside vendor presence; a dissatisfied incumbent can raise opportunity above a no-vendor account.
BELIEF-01	EXECUTIVE_BELIEF_TIMELINE records an exec's public-statement themes over time (sourced, dated) and derives a belief trajectory + conviction trend.
BELIEF-02	Trajectories ground message timing/angle and predict buying direction; they never infer private psychology of a named person (EPIS-ETH-01).
POWER-01	SIG-POWER derives an Influence Score (distinct from title) from recurring names, committee/board overlap, transformation sponsorship and decision recurrence — public signals only.
POWER-02	Shadow power centers (e.g. tech-subsidary CEO, transformation-office head, board advisor) are surfaced as decision influencers even when the org chart names someone else; flagged ~inferred until corroborated.
TENSION-01	SIG-TENSION detects competing priorities, conflicting public statements, contradictory initiatives and budget conflicts, outputting an Organizational Friction Score.
TENSION-02	Friction Score informs predicted deal velocity and sequencing; high friction triggers a strategy that addresses the resisting faction, not just the champion.
TENSION-03	Tension intelligence is built strictly from public, sourced signals; unsourced internal-politics inference is excluded from automated action (EPIS-ETH-02).
MOBILITY-01	SIG-MOBILITY tracks executive/board/consultant/vendor movement across the target network and emits Opportunity Transfer Events when a relationship moves between accounts.
MOBILITY-02	An Opportunity Transfer Event re-anchors SIG-PATH (follow-the-person), transfers relevant dossier knowledge to the new account, and is treated as a high-value buying predictor.
MOBILITY-03	Cross-account knowledge transfer respects per-account exclusivity and confidentiality boundaries; confidential per-account facts are not leaked across the network.
RELEVANCE-01	Every intelligence item receives a Strategic Relevance Score against four axes: relevance to Decimal solutions, to the account's current initiatives, to the current narrative, and to the current opportunity.
RELEVANCE-02	Low-relevance items are filtered from the active model (retained in Market Memory) so noise does not dilute reasoning or scoring.
RELEVANCE-03	Relevance is computed before an item feeds hypotheses or scoring; irrelevant facts cannot inflate an account's activity profile.

Rule ID	Specification
REASON-RCM-01	Every reasoning-stream output (SIG-HYP/ACCESS/VSAT/BELIEF/POWER/TENSION/MOBILITY/RELEVANCE) is governed by EPIS: RCM-stamped, coverage-capped, decay-aware, and withheld when confidence is below threshold.
REASON-RCM-02	The causal model per account is itself a versioned object carrying an aggregate Reality Confidence; recommendations cite which hypotheses and influencers they rest on, so a human can inspect the reasoning chain.

4.2 INT-ENR — Enrichment — Apollo · Clearbit · Clay

Enrichment resolves accounts and buying committees into verified, reachable contacts. The FSD mandates verified email enrichment only; INT-ENR specifies the provider chain, cost gating and conflict-merge that deliver it.

Why #1: Why use a fallback chain across three providers rather than one?

Because no single enrichment source covers the KSA banking ecosystem completely; a verified email missing in Apollo may exist in Clearbit or be inferable via Clay. A single provider produces unreachable contacts and bounce risk.

Why #2: Why gate enrichment on cost / eligibility rather than enrich everything?

Because enrichment calls cost money and rate-limit budget; enriching COLD/unreachable accounts wastes both. Enrichment is a scarce resource allocated by tier and reachability, consistent with Scarcity Economics.

Why #3: Why is verified-email-only non-negotiable?

Because an unverified email both bounces (damaging sender reputation, which INT-EML depends on) and pollutes the contact state machine with false REACHABLE statuses.

Why #4: Why specify a conflict-merge policy across providers?

Because providers disagree (different titles, stale roles); without a deterministic merge the engine stores contradictory contact facts and may address the wrong person.

Why #5: Why feed enrichment freshness back into scoring?

Because a stale contact record silently degrades reachability; freshness must be visible to scoring so an account isn't prioritized on the basis of a contact that no longer exists.

Root cause: Enrichment is a cost-gated, verified-only, fallback-chained resolution step whose freshness and conflicts must be explicit, never silently trusted.

New rules

Rule ID	Specification
INT-ENR-01	Provider fallback chain is ordered Apollo → Clearbit → Clay; resolution stops at the first verified result. Order is configurable but the verified-only gate is not.
INT-ENR-02	Only verified emails make a contact email-eligible; unverified → LinkedIn-only outreach path (per FSD ENR decision logic).
INT-ENR-03	Enrichment is gated by tier + reachability: HOT/WARM and path-reachable accounts enrich first; COLD/unreachable are deferred.
INT-ENR-04	Cross-provider conflict-merge: most-recent + highest-source-trust wins; unresolved title/role conflicts are flagged ~ and excluded from automated send.
INT-ENR-05	Every enriched record carries source + timestamp + freshness TTL; on expiry the contact is re-enriched before reuse.
INT-ENR-06	Buying-committee coverage is required: single-threaded enrichment (one contact per account) is flagged as insufficient coverage to scoring.
INT-ENR-07	Enrichment cost/volume is budgeted and logged; spend per account is capped and visible to the Scarcity (capital) layer.

4.3 INT-LIN — LinkedIn Automation — Heyreach

Heyreach executes LinkedIn connection requests and DMs. LinkedIn aggressively bans automation that breaches its platform policy, so INT-LIN is governed by hard caps and a ban-risk circuit breaker.

Why #1: Why hard-cap daily LinkedIn actions per sending account?

Because LinkedIn detects and bans accounts that exceed human-plausible action volumes; a banned account destroys an irreplaceable warm channel and any SIG-PATH P1 edges anchored on it.

Why #2: Why a circuit breaker rather than just a rate limit?

Because ban risk is non-linear: warning signs (restricted views, captcha challenges, withdrawal spikes) precede a ban. A breaker halts the account on early signals before the irreversible ban, which a static rate limit cannot.

Why #3: Why coordinate LinkedIn with email rather than run them independently?

Because simultaneous multi-channel contact reads as spam to the prospect and damages experience; the FSD mandates coordinated, not concurrent, channel use.

Why #4: Why tie LinkedIn sending accounts to SIG-PATH?

Because the strongest path (P1/P2) often runs through a specific Decimal person's LinkedIn; the sending account must match the warm path, not be assigned round-robin.

Why #5: Why treat a connection-accept as a state transition, not just a metric?

Because acceptance changes what outreach is permitted (DM eligible) and feeds reply detection / handoff; it must update the contact state machine, not merely increment a counter.

Root cause: LinkedIn automation must operate inside platform-policy-safe caps with an early-warning circuit breaker, warm-path-matched sending accounts, and coordinated (never concurrent) channel timing — because a banned account is an irreversible loss of warm capital.

New rules

Rule ID	Specification
INT-LIN-01	Per-account daily caps on connection requests + DMs + profile views are enforced below LinkedIn's human-plausible thresholds; caps are configurable downward only.
INT-LIN-02	Ban-risk circuit breaker: on early-warning signals (captcha challenge, view restriction, accept-rate collapse, withdrawal spike) the account is auto-paused and flagged for human review.
INT-LIN-03	Channels are coordinated, not concurrent: email and LinkedIn touches to the same contact are spaced per sequence design; simultaneous same-day multi-channel contact is blocked.
INT-LIN-04	Sending-account assignment matches the resolved SIG-PATH where a P1/P2 Decimal-person path exists; round-robin only for P4/P5 cold paths.
INT-LIN-05	Connection-accept and DM-reply update the contact state machine (→ OPENED/REPLIED) and trigger reply-detection + handoff logic.
INT-LIN-06	Account warmup: new sending accounts ramp action volume gradually before reaching standard caps.
INT-LIN-07	A banned/restricted account immediately emits SIG-PATH path-break events for every P1 edge it anchored.

4.4 INT-EML — Email Execution & Deliverability

The FSD treats deliverability as a strategic layer, not a technical afterthought: enterprise outreach succeeds or fails on inbox placement and sender reputation. INT-EML specifies the infrastructure and gating that protect it.

Why #1: Why mandate SPF, DKIM and DMARC before any sending?

Because without authentication, enterprise mail filters route outreach to spam regardless of content quality; the entire personalization investment is wasted at the gateway.

Why #2: Why gate sending on domain warmup?

Because a cold domain sending at volume is the strongest spam signal; warmup builds the sender reputation that inbox placement depends on.

Why #3: Why throttle volume and rotate sending identities?

Because spikes from a single identity trip volume-based spam heuristics; low-volume, rotated, high-relevance sending is the FSD's mandated posture (low volume / high personalization, never mass blast).

Why #4: Why feed bounce/spam telemetry back as a reputation signal?

Because reputation degrades silently; without a feedback loop the engine keeps sending into a damaged reputation, compounding the damage. Telemetry must gate further sending.

Why #5: Why couple email reputation to enrichment (INT-ENR) verification?

Because the fastest way to destroy reputation is sending to unverified addresses that bounce; verified-only enrichment is a precondition of deliverability, linking the two domains.

Root cause: Email is a reputation-managed channel: authenticated, warmed, volume-throttled, rotation-balanced, and reputation-feedback-gated — with verified-only recipients as a hard precondition.

New rules

Rule ID	Specification
INT-EML-01	SPF, DKIM and DMARC must be configured and validated per sending domain before that domain is permitted to send.
INT-EML-02	Domain warmup is mandatory: new domains ramp volume on a schedule and cannot send at full volume until warmed.
INT-EML-03	Per-domain volume throttles + sending-identity rotation enforce a low-volume, high-relevance posture; mass blasting is structurally blocked.
INT-EML-04	Bounce, complaint and spam-placement telemetry feed a per-domain reputation score; below a floor, the domain is auto-paused from sending.
INT-EML-05	Only INT-ENR-verified emails may be sent to; unverified addresses are never sent email (LinkedIn-only path).
INT-EML-06	Reply received pauses the sequence and updates contact state (→ REPLIED) per FSD EXEC-002.
INT-EML-07	Duplicate-outreach prevention: the same contact is never sent the same touch twice across workflows (FSD EXEC-003).
INT-EML-08	Arabic / English send selection follows the contact's resolved language preference; deliverability rules apply identically to both.

4.5 INT-CRM — CRM Synchronization — HubSpot

After positive engagement, sales takes over and needs full context in the system of record. INT-CRM specifies a sync that cannot corrupt that system of record — the failure mode that destroys trust in the whole engine.

Why #1: Why sync engagement history to the CRM at all?

Because a cold open from sales destroys a warm signal; sales needs the full engagement context to take over without re-pitching, and the CRM is the system they already trust.

Why #2: Why split authority between ABM and CRM rather than one source of truth?

Because dual authority over the same field causes write-conflicts; ABM is authoritative for signals/scores, CRM for deal stage/owner. No field may be authoritative in both.

Why #3: Why must sync be idempotent?

Because retries (network, queue) re-deliver the same event; without idempotency the CRM accumulates duplicate tasks/activities, polluting the record sales relies on.

Why #4: Why route failed syncs to a dead-letter queue rather than retry forever or drop?

Because a silently dropped sync leaves ABM and CRM disagreeing on account state — and a disagreement can cause automation to re-engage an account sales has already closed. A DLQ makes failures visible and recoverable.

Why #5: Why have CRM deal-stage win conflicts and pause automation?

Because the human-owned deal stage reflects ground truth that the engine cannot see; when they conflict, the engine must defer to the human and stop automated outreach to avoid contradicting sales.

Root cause: CRM sync must be idempotent, authority-split, conflict-resolved in favour of the human-owned deal stage, and dead-lettered on failure — because corrupting the system of record is the one failure that discredits the engine.

New rules

Rule ID	Specification
INT-CRM-01	Sync is idempotent on (account_id, event_hash); re-delivery of an already-synced event is a no-op.
INT-CRM-02	Authority split: ABM is authoritative for signals/scores/engagement; CRM is authoritative for deal stage/owner. No field is dual-authoritative.
INT-CRM-03	Failed syncs route to a dead-letter queue with full payload + error; never a silent drop.
INT-CRM-04	On state conflict, CRM deal-stage wins and automated outreach to that account is paused pending human review.
INT-CRM-05	Reply / positive engagement triggers: pause automation → assign owner → generate AI meeting brief → Slack notify → create CRM task (FSD handoff sequence).
INT-CRM-06	CRM credentials are encrypted secrets; sync runs under least-privilege scopes (FSD security architecture).

4.6 INT-SLK — Notifications — Slack

Slack delivers the human-handoff alert and AI meeting brief to the sales team at the decisive moment. INT-SLK specifies a reliable, rate-safe, well-formed notification contract.

Why #1: Why push handoff notifications to Slack rather than only the CRM task?

Because timing is the competitive weapon: a positive reply decays fast, and a Slack ping reaches the owner in real time where a CRM task waits to be noticed.

Why #2: Why define a strict brief payload contract?

Because an underspecified alert forces the rep back into research, losing the time advantage; the brief must carry enough grounded context (account, signal, path, suggested angle) to act immediately.

Why #3: Why rate-limit Slack notifications?

Because alert floods cause the team to ignore them (alert fatigue), defeating the purpose; notifications must be deduplicated and prioritized.

Why #4: Why require delivery confirmation?

Because a silently failed notification means a hot handoff is never picked up; delivery must be confirmed or the alert escalated/retried.

Root cause: Slack notifications must be real-time, contract-formed with a grounded brief, rate-limited against fatigue, and delivery-confirmed so no hot handoff is silently lost.

New rules

Rule ID	Specification
INT-SLK-01	Handoff notification fires in real time on the engagement trigger, carrying the AI meeting brief.
INT-SLK-02	Brief payload contract: account, triggering signal, resolved warm path/owner, and suggested angle — all grounded in INT-SIG facts.
INT-SLK-03	Notifications are deduplicated and priority-ranked; volume is rate-limited to prevent alert fatigue.
INT-SLK-04	Delivery is confirmed; an unconfirmed/failed notification is retried then escalated, never silently dropped.

4.7 INT-ANL — Analytics — Metabase

Analytics are not only reporting — the FSD uses them for system learning, signal optimization and prompt refinement. INT-ANL specifies a safe, read-only analytics boundary.

Why #1: Why connect Metabase to a read replica, never the primary?

Because analytical queries are heavy and unpredictable; running them on the primary risks degrading the live ingestion/scoring pipeline.

Why #2: Why forbid analytics write-back into operational tables?

Because analytics is a consumer, not a producer, of operational truth; letting dashboards write back creates an uncontrolled path that can corrupt scoring inputs.

Why #3: Why define an explicit KPI contract?

Because the analytics layer feeds the optimization loop (scoring weights, prompt refinement); ambiguous metric definitions silently mis-tune the engine.

Why #4: Why route optimization changes through a controlled loop rather than direct dashboard edits?

Because changing scoring weights or prompts is a model-governance action; it must be reviewed and versioned, not applied live from a BI tool.

Root cause: Analytics is a read-only, contract-defined consumer whose insights feed a governed optimization loop — never a direct write path into the operating engine.

New rules

Rule ID	Specification
INT-ANL-01	Metabase reads from a read replica only; no analytical query touches the primary operational database.
INT-ANL-02	Analytics has no write access to operational tables; it is strictly read-only.
INT-ANL-03	KPI definitions are contracted and versioned (signal, persona, messaging, conversion performance) so the optimization loop tunes on stable metrics.
INT-ANL-04	Optimization outputs (weight/prompt changes) flow through a governed, reviewed, versioned loop — not applied directly from the BI layer.

4.8 INT-ORC — Orchestration — n8n

n8n is the orchestration layer, event router and integration engine. The FSD is explicit: n8n must NOT contain business logic, scoring logic, or AI reasoning — those belong in APIs/services. INT-ORC enforces that boundary.

Why #1: Why keep business/scoring/AI logic out of n8n?

Because logic embedded in workflows is untestable, unversioned and invisible to governance; the same rule could drift across workflows. Logic belongs in versioned services with a single source of truth.

Why #2: Why make every workflow step idempotent?

Because n8n retries and re-runs steps on failure; non-idempotent steps double-send outreach or double-write CRM records on retry.

Why #3: Why group workflows (signal / intelligence / execution / analytics)?

Because grouping isolates failure domains and clarifies ownership; a failure in execution workflows shouldn't halt signal ingestion.

Why #4: Why standardize the

Trigger→Validation→Decision→Execution→Logging→State-update pattern?

Because a uniform step pattern guarantees every workflow validates inputs, logs, and updates state consistently — the observability and recovery layers depend on that uniformity.

Why #5: Why route all external calls through n8n rather than service-to-service?

Because centralizing integration in the orchestration layer gives one place for retry, rate-limit and observability policy, instead of scattering it across services.

Root cause: n8n is a logic-free orchestration layer of idempotent, uniformly-patterned, grouped workflows — the integration and retry/observability choke point, with all business logic delegated to versioned services.

New rules

Rule ID	Specification
INT-ORC-01	n8n contains orchestration only: no business logic, no scoring logic, no AI reasoning. These live in versioned APIs/services.
INT-ORC-02	Every workflow step is idempotent; retries and re-runs cannot double-send outreach or double-write records.
INT-ORC-03	Workflows are grouped into Signal / Intelligence / Execution / Analytics domains with isolated failure boundaries.
INT-ORC-04	Every workflow follows the standard pattern: Trigger → Validation → Decision → Execution → Logging → State update.
INT-ORC-05	All external integration calls route through n8n so retry, rate-limit and observability policy are applied in one place.
INT-ORC-06	Workflow execution, decisions and errors are logged to the observability layer (OBS) on every run.
INT-ORC-07	n8n calls services over the defined API contracts (4.9); it never reaches into service databases directly.

4.9 API — Service Contracts & Interface Design

The engine exposes modular APIs (FastAPI) for scoring, enrichment, AI generation, CRM sync and analytics, enabling modular orchestration and future frontend support. 4.9 specifies the contracts.

Why #1: Why expose modular APIs rather than a monolith?

Because modular services allow independent scaling, isolation and a future frontend; they also give n8n clean contracts to orchestrate against without embedding logic.

Why #2: Why version every API contract?

Because the engine and its orchestration evolve independently; unversioned contracts break workflows silently on change. Versioning lets producers and consumers evolve safely.

Why #3: Why authenticate and scope every endpoint?

Because these services handle enterprise contact data and banking context; an unauthenticated or over-scoped endpoint is a data-exposure path (PDPL exposure).

Why #4: Why rate-limit service endpoints internally?

Because a runaway workflow or retry storm can overwhelm a service (especially AI generation and enrichment, which are costly); internal rate limits protect availability and cost.

Why #5: Why define explicit request/response schemas and error contracts?

Because the retry/DLQ layer (4.10) keys its recovery strategy on error type; without a stable error contract the failure architecture cannot distinguish a retryable timeout from a permanent rejection.

Root cause: Every service is a versioned, authenticated, scoped, rate-limited contract with explicit request/response and error schemas — the stable surface the orchestration and failure layers depend on.

New rules

Rule ID	Specification
API-01	Core endpoints: POST /signals (store normalized signal), POST /score-account (dynamic score), POST /generate-message (outreach), POST /sync-hubspot (CRM sync), plus analytics read APIs.
API-02	Every endpoint is versioned (e.g. /v1/...); breaking changes ship under a new version, never mutate an existing contract.
API-03	Every endpoint requires authentication and runs under least-privilege scopes; no anonymous access to contact/banking data.
API-04	Explicit request/response JSON schemas are published per endpoint; inputs are validated and rejected with a typed error on violation.
API-05	A stable error contract distinguishes retryable (timeout, 5xx, rate-limited) from permanent (validation, auth) failures for the FAIL layer.
API-06	Internal rate limits protect costly services (/generate-message, enrichment) from retry storms and runaway workflows.
API-07	Secrets/credentials are never passed in URLs or query strings; only in authenticated headers/body (privacy rule).
API-08	/score-account returns score + tier + the contributing factors (signal strength, regulatory pressure, reachability, relationship warmth) for explainability.
API-09	/generate-message accepts only grounded inputs from INT-SIG and rejects ungrounded personalization (hallucination gate).
API-10	n8n is the only orchestration caller; services do not call each other's databases, only their published APIs.

4.10 FAIL — Retry, Dead-Letter & Circuit-Breaker Architecture

The FSD specifies per-failure recovery, retry queues, dead-letter queues and circuit breakers. 4.10 consolidates these into one failure-recovery matrix so no failure is silent or unrecoverable.

Why #1: Why match recovery strategy to failure type rather than retry everything uniformly?

Because failures differ: a timeout is transient (retry), a validation error is permanent (don't retry), a CRM conflict needs human review (DLQ). Uniform retry either gives up too early or loops forever.

Why #2: Why dead-letter rather than drop or infinite-retry?

Because a dropped failure corrupts state silently and an infinite retry wedges the pipeline; a DLQ preserves the payload for inspection and replay while letting the pipeline proceed.

Why #3: Why exponential backoff specifically?

Because immediate aggressive retries against a struggling dependency worsen the outage (retry storm); backoff gives the dependency room to recover.

Why #4: Why circuit breakers on top of retries?

Because some failures (LinkedIn ban-risk) are made worse by retrying; the breaker stops the action entirely on early-warning signals rather than retrying into an irreversible ban.

Why #5: Why make every failure observable (no silent failure)?

Because the worst failure is the invisible one — it corrupts state and is discovered only when an account is mis-handled. Every failure must be logged, counted and surfaced.

Root cause: Failure handling is type-matched: transient → backoff retry, permanent → typed rejection, stateful-conflict → dead-letter, ban-risk → circuit-break — and every failure is observable, never silent.

Failure type	Recovery strategy	Visibility
API timeout	Exponential backoff retry	Logged; alert on repeated
AI generation failure	Retry with fallback prompt	Logged + QC flag
Email send failure	Queue retry	Reputation telemetry
CRM sync failure	Dead-letter queue	DLQ visible + alert
Enrichment failure	Deferred processing	Freshness flag to scoring
LinkedIn ban-risk signal	Circuit-breaker pause	Human-review flag + path-break

New rules

Rule ID	Specification
FAIL-01	Recovery strategy is selected by failure type per the matrix; the API error contract (API-05) supplies the classification.
FAIL-02	Transient failures (timeout, 5xx, rate-limited) use exponential backoff retry with a max-attempt cap.
FAIL-03	Permanent failures (validation, auth) are not retried; they return a typed error and are logged.
FAIL-04	Stateful-integration failures (CRM sync) dead-letter with full payload for inspection and replay (links to INT-CRM-03).
FAIL-05	AI generation failures retry with a fallback prompt then escalate to QC/human, never emit ungrounded output.
FAIL-06	Enrichment failures defer processing and flag contact freshness to scoring rather than blocking the pipeline.
FAIL-07	LinkedIn ban-risk and email reputation-floor breaches trigger circuit-breaker pauses (links INT-LIN-02 / INT-EML-04), not retries.
FAIL-08	No failure is silent: every failure is logged, counted, and alerts on repeated/threshold breach.
FAIL-09	Dead-letter and circuit-breaker states are human-reviewable and replayable; recovery actions are audited.
FAIL-LEARN-01	Every failure generates a Failure Pattern Record: root cause, precursor signals, recovery effectiveness, and prevention strategy — failures are treated as lessons, not just events.

Rule ID	Specification
FAIL-LEARN-02	Failure Pattern Records form a searchable organizational memory; recurring precursor patterns (e.g. signals preceding a LinkedIn ban) become predictive so the engine can pre-empt failures before they occur.

4.11 OBS — Observability, Security & Compliance

The platform handles enterprise contact data and banking context, and must remain aligned with PDPL, email compliance and LinkedIn platform policy. OBS specifies what is logged, secured and governed.

Why #1: Why log workflow execution, AI prompts/outputs, API failures and engagement events?

Because the engine makes autonomous decisions; without a full execution + reasoning trail, failures can't be debugged and decisions can't be explained or audited.

Why #2: Why log AI prompts and outputs specifically?

Because hallucination and metric-manipulation failure modes are only detectable by inspecting what the AI was asked and what it produced; the prompt/output log is the audit surface for AI governance.

Why #3: Why enforce role-based access + audit trails on this data?

Because enterprise contact and banking-context data is sensitive (PDPL); access must be least-privilege and every access auditable to demonstrate compliance.

Why #4: Why hold secrets in environment variables / encrypted credentials, never in code or URLs?

Because leaked credentials expose the CRM and enrichment data wholesale; secrets handling is the base of the security model and a hard privacy rule.

Why #5: Why make PDPL/compliance a continuous alignment requirement rather than a one-time check?

Because outreach scope, data retention and platform policies evolve; compliance must be a standing constraint the engine operates within, consistent with the Constraint Hierarchy's top tier (Legal/Compliance).

Root cause: Observability and compliance are a standing top-tier constraint: full execution + AI-reasoning logging, least-privilege audited access, encrypted secrets, and continuous PDPL/platform-policy alignment.

New rules

Rule ID	Specification
OBS-01	Mandatory logging: workflow execution, AI prompts, AI outputs, API failures, sequence activity, engagement events.
OBS-02	AI prompt/output logs are retained as the audit surface for hallucination and metric-manipulation detection.
OBS-03	Role-based access control + audit trails on all contact/banking data; access is least-privilege and recorded.
OBS-04	Secrets live in environment variables / encrypted credential stores; never in code, logs, or URLs.
OBS-05	Outreach data is restricted-access; CRM sync uses encrypted credentials (FSD security controls).
OBS-06	The platform operates within PDPL, email-compliance and LinkedIn platform-policy constraints as a continuous, top-tier (Legal/Compliance) requirement.
OBS-07	Compliance constraints sit at the top of the Constraint Hierarchy: no optimization, timing or capital rule may override them.
DEPENDENCY-01	Every integration is assigned a Criticality Level: Tier A (system cannot function without it), Tier B (degraded operation), Tier C (optional). Criticality drives failover priority and alerting severity.
DEPENDENCY-02	A maintained Dependency Graph maps which modules degrade or fail if a given integration fails; correlated/simultaneous failures (e.g. Apollo outage + LinkedIn restriction + HubSpot latency) are modelled and alerted as compound risk, not just isolated incidents.

4.12 SECTION 4 RULE ROLL-UP & CROSS-LINKS

Domain	New rules	Key cross-links into Sections 1–3
EPIS	18	Principle 3 uncertainty reduction, Module 0 hallucination governance, Constraint Hierarchy (Legal/PDPL), Scarcity (value-of-information)
INT-SIG (acq.)	34	Scoring (Effective Opportunity), Market Memory, Timing windows, Scarcity (attention)
INT-SIG (reason.)	23	Strategic Reasoning (Principle 5), Effective Opportunity (displacement), SIG-PATH (access/mobility)
INT-ENR	7	Scarcity Economics (cost gating), reachability scoring
INT-LIN	7	SIG-PATH (warm-path matching, path-break), Timing (channel coordination)
INT-EML	8	Deliverability as strategic layer, contact state machine
INT-CRM	6	Human handoff (Principle 3), system-of-record integrity
INT-SLK	4	Timing as competitive weapon, grounded brief (Principle 5)
INT-ANL	4	Optimization loop, model governance
INT-ORC	7	Constraint Hierarchy (logic single-source-of-truth), idempotency
API	10	Explainability (Principle 3), hallucination gate (Module 0)
FAIL	11	Failure-mode-as-first-class-concern; failure learning → predictive prevention
OBS	9	Constraint Hierarchy top tier (Legal/Compliance), AI governance, dependency graph

SECTION 4 COMPLETE — WHAT IT ESTABLISHED

Section 4 specified the engine's external boundary AND the epistemic spine that governs it. The EPIS layer and its Reality Confidence Model make explicit, per fact and per recommendation, how confident the engine is and what would change its mind — separating source reliability from incentive-bias, capping confidence by coverage, gating acquisition by value-of-information, decaying beliefs on category-specific half-lives, and measuring trust per subsystem. INT-SIG was reframed from an intelligence repository into an evolving causal model: competing hypotheses, access (not just connection), incumbent vulnerability, belief trajectories, real influence, internal friction, network mobility, and strategic relevance — every output RCM-governed and bounded to public signals.

~152 new rules, additive to the Section 1–3 base. The shift is from *aggregating intelligence* to *deciding what reality to trust* — the deepest layer underneath Section 4.

ON THE HORIZON

Section 5 — Edge Cases & Failure Modes (cross-cutting scenarios that span modules + integrations), and the eventual consolidation of all section PDFs into the unified master specification.